Python Notes (Lecture 4)

Dictionaries

1. Dictionary in Python

Dictionaries store data in **key : value** pairs, written inside curly brackets `{ }`.

- **Mutable**: you can change, add or remove items.
- **No duplicate keys**: if a key repeats, the latest value replaces the old one.
- Values can be of any type: string, number, list, even another dictionary.

```
info = {
    "name": "shradha",
    "cgpa": 9.6,
    "marks": [98, 97, 95],
}

print(info["name"])
print(info["cgpa"])
print(info["marks"])
```

Output:

```
shradha
9.6
[98, 97, 95]
```

Use `dict["key"] = value` to **change** an existing value or **add** a new pair:

```
info = {"name": "shradha", "cgpa": 9.6}

info["cgpa"] = 9.8          # changes the value
info["city"] = "Delhi"      # adds a new key
print(info)
```

Output:

```
{'name': 'shradha', 'cgpa': 9.8, 'city': 'Delhi'}
```

Tip: avoid naming a variable `dict`, because that hides Python's built-in `dict()` function.

---

2. Nested Dictionaries

A dictionary can be stored as a value inside another dictionary.

```
student = {
    "name": "shradha",
    "score": {
        "chem": 98,
        "phy": 97,
        "math": 95,
    },
}

print(student["score"])
print(student["score"]["math"])
```

Output:

```
{'chem': 98, 'phy': 97, 'math': 95}
95
```

---

3. Dictionary Methods

Method	What it does
`d.keys()`	Returns all keys
`d.values()`	Returns all values
`d.items()`	Returns all (key, value) pairs as tuples
`d.get("key")`	Returns the value for the key (gives `None` instead of an error if the key is missing)
`d.update(new_dict)`	Inserts the items of new_dict into the dictionary

```python
student = {"name": "shradha", "cgpa": 9.6}

print(student.keys())
print(student.values())
print(student.items())
print(list(student.keys()))     # convert to a normal list
```

Output:

```
dict_keys(['name', 'cgpa'])
dict_values(['shradha', 9.6])
dict_items([('name', 'shradha'), ('cgpa', 9.6)])
['name', 'cgpa']
```

```python
student = {"name": "shradha", "cgpa": 9.6}

print(student.get("name"))
print(student.get("age"))      # key missing: returns None
# print(student["age"])        # key missing: KeyError

student.update({"city": "Delhi", "cgpa": 9.8})
print(student)
```

Output:

```
shradha
None
{'name': 'shradha', 'cgpa': 9.8, 'city': 'Delhi'}
```

Sets

1. Set in Python

A set is a collection of **unordered** items written in curly brackets `{ }`.

- Each element must be **unique**: repeated values are stored only once.
- Each element must be **immutable** (numbers, strings, tuples). Lists cannot be stored in a set.
- Sets have no index, so `nums[0]` does not work.

```python
nums = {1, 2, 3, 4}
set2 = {1, 2, 2, 2}      # repeated elements stored only once

print(nums)
print(set2)
print(len(set2))
```

Output:

```
{1, 2, 3, 4}
{1, 2}
2
```

An empty set is created with `set()`. Writing `{}` creates an empty **dictionary**, not a set.

```python
null_set = set()
print(type(null_set))
print(type({}))
```

Output:

```
<class 'set'>
<class 'dict'>
```

2. Set Methods

Method	What it does
`s.add(el)`	Adds an element
`s.remove(el)`	Removes the element (error if it is not in the set)
`s.clear()`	Empties the set
`s.pop()`	Removes and returns a random value
`s.union(set2)`	Combines the values of both sets and returns a new set
`s.intersection(set2)`	Returns a new set with only the common values

```
nums = {1, 2, 3}

nums.add(4)
nums.add(2)         # already present: no change
print(nums)

nums.remove(3)
print(nums)

nums.clear()
print(nums)
```

Output:

```
{1, 2, 3, 4}
{1, 2, 4}
set()
```

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}

print(set1.union(set2))
print(set1.intersection(set2))
```

Output:

```
{1, 2, 3, 4}
{2, 3}
```

Let's Practice

Q1. Store the following word meanings in a Python dictionary.

table : "a piece of furniture", "list of facts & figures"
cat : "a small animal"

```
dictionary = {
    "table": ["a piece of furniture", "list of facts & figures"],
    "cat": "a small animal",
}

print(dictionary)
```

Output:

```
{'table': ['a piece of furniture', 'list of facts & figures'], 'cat': 'a small animal'}
```

"table" has two meanings, so we store them in a list.

Q2. Count the classrooms needed for the subjects below (one classroom per subject).

"python", "java", "C++", "python", "javascript", "java", "python", "java", "C++", "C"

```
subjects = {"python", "java", "C++", "python", "javascript",
            "java", "python", "java", "C++", "C"}

print("Classrooms needed:", len(subjects))
```

Output:

```
Classrooms needed: 5
```

A set keeps only unique subjects, so its length is the number of classrooms.

Q3. Enter marks of 3 subjects from the user and store them in a dictionary.

Start with an empty dictionary and add one by one. Use subject name as key and marks as value.

```
marks = {}

marks["phy"] = int(input("Enter phy marks: "))
marks["chem"] = int(input("Enter chem marks: "))
marks["math"] = int(input("Enter math marks: "))

print(marks)
```

Output:

```
Enter phy marks: 95
Enter chem marks: 88
Enter math marks: 97
{'phy': 95, 'chem': 88, 'math': 97}
```

Q4. Store 9 and 9.0 as separate values in a set.

Python treats `9` and `9.0` as equal, so a set keeps only one of them:

```
values = {9, 9.0}
print(values)
```

Output:

```
{9}
```

To keep both, store each as a tuple that also records its type:

```
values = {("int", 9), ("float", 9.0)}
print(len(values))
```

Output:

```
2
```